

CheckPoint 4

1. ¿Cuál es la diferencia entre una lista y una tupla en Python?

Ambas son colecciones de elementos que pueden ser de igual o diferente tipo.

La principal diferencia es que una lista se puede modificar y una tupla no.

Una lista se declara entre corchetes [], mientras que para una tupla se utilizan paréntesis ()

Si una tupla tiene un elemento mutable, como una lista, este elemento sí se puede modificar, lo que no se puede es cambiar el elemento en sí.

Las tuplas son más rápidas y ocupan menos espacio en memoria ya que no necesitan reservar espacio adicional para futuras modificaciones.

Uso común:

- Listas:

Para conjuntos de datos que necesitan cambiar (ordenar, añadir, eliminar) y suelen ser del mismo tipo, aunque no es obligatorio, para facilitar operaciones uniformes sobre todos sus elementos, tales como cálculos, ordenación o búsquedas.

Ejemplo:

- Lista:

```
lista = ['Uno', 2, 'Tres', ['A', 'B', 'C', 'D'], 'Cinco']
```

```
lista[1] = 'Dos'
```

```
lista.sort()
```

Checkpoint 4

- Tuplas:

Para datos constantes o heterogéneos que no deben cambiar; esto implica varias cosas:

- Inmutabilidad y seguridad.

Al no poder modificarse (no se pueden añadir, borrar ni reasignar sus elementos), garantiza que los valores permanecen iguales durante la ejecución de la aplicación, y esto es especialmente útil cuando no quieres que estos elementos puedan ser modificados accidentalmente.

Ejemplo:

```
tupla = ('Uno', 2, 'Tres', ['A', 'B', 'C', 'D'], 'Cinco')
```

`tupla[1] = 'Dos'` → esto NO es correcto

`tupla[3] = 20` → esto NO es correcto

`tupla[3][1] = 'J'` → esto es correcto

- Indica al programador de forma simple que esos datos no se deben alterar.
- Como puede contener datos de distintos tipos la hace práctica para representar un registro pequeño con campos diferentes, por ejemplo id, nombre, valoración...
- Se puede usar como clave para un diccionario

```
registro = {  
    (2021, 'A021') : [30000, 'beneficios'],  
    (2022, 'A022') : [10000, 'perdidas'],  
}
```

Checkpoint 4

2. ¿Cuál es el orden de las operaciones?

Las operaciones se ordenan de la siguiente forma:

- Paréntesis
 - Exponente
 - Multiplicación
 - División
 - Suma
 - Resta
- } El orden es intercambiable
- } El orden es intercambiable

Creo que la mejor manera de entenderlo es con un ejemplo:

$$3(5-2+3^2)/6$$

Lo primero es resolver lo que está entre paréntesis empezando por el exponente

$$3*(5-2+9)/6$$

Seguimos con la suma y la resta, que a pesar de que deberían ser las últimas operaciones a realizar, al estar entre paréntesis tienen prioridad

$$3*(12)/6$$

Después, cómo la multiplicación y la división tienen el mismo orden de prioridad las realizaremos de izquierda a derecha.

$$36/6$$

Y ya sólo queda por hacer la división

$$36/6=6$$

Checkpoint 4

3. ¿Qué es un diccionario Python?

Es una estructura de datos similar a una lista con el formato clave-valor.

Así como en una lista se utiliza el índice para acceder a un elemento, en un diccionario se utiliza una clave, que es única (no puede repetirse) para acceder al valor del elemento.

Al igual que la lista, también se puede modificar.

Se declara entre llaves {} y los elementos se separan por comas ','.

```
diccionario= {  
    'primero': 1,  
    'segundo': 'dos',  
    'tercero': 3.5,  
    'cuarto': ['A', 'B', 'C', 'D'],  
}
```

Para acceder a sus elementos se utiliza su clave.

```
print(diccionario['cuarto']) → ['A', 'B', 'C', 'D']
```

```
print(diccionario) → {'primero': 1, 'segundo': 'dos', 'tercero': 3.5,  
    'cuarto': 'String'}
```

Para modificar el valor de un elemento basta con asignarle otro.

```
diccionario['cuarto'] = 'String'
```

Para eliminar un elemento se puede usar el comando del o el método pop.

```
del(diccionario['2'])
```

```
diccionario.pop('3')
```

Checkpoint 4

4. ¿Cuál es la diferencia entre el método ordenado y la función de ordenación?

Método ordenado	Función de ordenación
Es un método exclusivo de las listas	Es una función de Python
Modifica la lista original	No modifica el objeto original.
No devuelve un valor	Crea una nueva lista ordenada.
Ambos admiten dos argumentos opcionales Key y reverse. Key tiene el valor de la función que será llamada en cada ítem de la lista, por ejemplo len(), que indica que la ordenación será por longitud. Reverse se iguala a True para indicar que se debe ordenar en orden inverso.	
Ventaja: Es ligeramente más eficiente en memoria ya que no crea una copia, es la opción ideal sin no es necesario mantener el orden original.	Ventaja: No sólo funciona con listas, también es aplicable a tuplas, diccionarios y conjuntos.
Ejemplo: <pre>nombres = ['Jessica', 'Ben', 'Carl', 'Jackie', 'Wendy']</pre>	
Uso: <pre>nombres.sort() nombres.sort(key=len) nombres.sort(reverse=True) nombres.sort(key=len, reverse=True)</pre>	Uso: <pre>nueva_lista=sorted(nombres) print(sorted(nombres, reverse=True)) nueva_lista=sorted(nombres, key=len) nueva_lista=sorted(key=len, reverse=True))</pre>

Checkpoint 4

5. ¿Qué es un operador de reasignación?

La asignación aumentada (reasignación) combina una operación binaria con una asignación en un solo paso. Toma una variable, le aplica una operación con otro valor y almacena el resultado nuevamente en la misma variable.

Operación	(X=10)	Operador de reasignación	Resultado
Suma	$x=x+3$	$x +=3$	13
Resta	$x=x-3$	$x -=3$	7
Multiplicación	$x=x*2$	$x *=2$	20
División	$x=x/2$	$x /=2$	5
Floor	$x=x//3$	$x //=3$	3
Módulo	$x=x\%3$	$x \%=3$	1
Exponente	$x=x**2$	$x **=2$	100